

得分	教师签名	批改日期

深圳技术大学实验报告

课程编号：HE00239

课程名称：自然语言处理

实验名称：实验1-NLP开发环境与基础文本处理

班 级：24智医三班

指导教师：张媛

报 告 人：陈佳豪 学 号：202400502133

合作者：无 组号：

实验地点：A2-314

实验时间：2026 年 10 月 8 日 星期四

提交时间：2026 年 10 月 10 日

一、实验目的

1. 掌握 Python 环境检查、脚本运行与文件路径的基本操作；
2. 掌握 UTF-8、UTF-8-SIG、GB18030、UTF-16 四种中文文本编码的识别与读取方法；
3. 掌握 Unicode 规范化（NFKC）、空白处理、隐私字段掩码与基础词频统计；
4. 会用中间输出和断言核验处理结果，让每一步都可观察、可复核；
5. 能完整记录“错误现象—原因定位—修改—复测”的调试过程。

二、实验原理

2.1 多编码探测：试错式解码与 BOM

中文文本落到磁盘上只是一串字节，编码信息不跟着文本走。用错编码去读同一串字节，会抛 `UnicodeDecodeError`。但没抛异常不等于读对了——`utf-16` 会把任意偶数长度的字节硬凑成对，几乎总能“读成功”，读出来是乱码。所以我按 `utf-8` → `utf-8-sig` → `gb18030` → `utf-16` 的顺序逐个试，把每次失败的编码和异常类型记进诊断列表。这个顺序不能调，`utf-16` 放在前面会把大多数文件变成“读错了却通过了”的假象。

BOM 要单独处理。带 BOM 的 UTF-8（也就是 UTF-8-SIG）字节序列本身仍是合法 UTF-8，用 `utf-8` 读不报错，只是首字符残留 `U+FEFF`。所以读成功之后还得看一眼首字符，命中就改用 `utf-8-sig` 重读一遍。这种“读成功了但数据是脏的”，比直接抛异常更难发现。

2.2 Unicode 规范化 NFKC

NFKC 把全角折成半角。C 组实测：`3 0` (`U+FF13/U+FF10`) → `30`，`—` (`U+FF0D`) → `-`；冒号 `:` (`U+FF1A`) 与 逗号 `,` (`U+FF0C`) 分别转成半角的 `:` 与 `,`。中文习惯的句号 `U+3002` 没有兼容分解，规范化后原样不动。

这一步是有损的，风险在洗得太干净。指导书明确禁止用“只保留汉字”的正则，否则 `128/78 mmHg`、`5 mg/片` 会被整段删掉。所以我只做 NFKC 和空白折叠两步，不删词。

2.3 隐私字段掩码

手机号正则 `(?<!\d)(1[3-9]\d{4}\d{4})(?!\\d)` 前后加了负向断言，免得更长的数字串中间截一段来打码，把无关数字误伤。邮箱保留前 2 个字符，方便人工核对字段归属，又不至于泄露完整地址。这里用 `re.subn` 而不是 `re.sub`，因为替换次数本身就是后面要断言的量。

2.4 规则切词、停用词与断言

切词正则把多字短语排在单字规则前面：`患者|建议|进行|不伴|否认|不得|无|未|英文|数字|单字`。候选分支按书写顺序尝试，「不伴」要是放到单字规则后面，就会被拆成“不”+“伴”，否定语义当场丢掉——这是医疗文本里最危险的一类错误。

还有个口径问题。指导书要求 `token` 里含“未”，所以“未见”必须拆成 `未 + 见`。我一开始想把“未见”当整词加进多字规则，试算下来 `token` 里就没有“未”了，

指导书自己的断言反而会失败，于是按指导书的口径写，理由留在代码注释里。

停用词这边，通用词表是拿新闻和社交语料训出来的，不含医疗否定语义。要是把无、未、不伴 当停用词删掉，「未见异常放电」会变成「见异常放电」，「无意识障碍」会变成「意识障碍」——一个是没有症状，一个是确诊有病。所以加载完词表要减去否定与约束保护词。本实验发的 stopwords.txt 共 8 个词（的、了、和、与、于、患者、建议、进行），本身没有否定词，保护逻辑在这次数据上不改变结果，但代码还是要写，换一份词表就会翻车（见 5.4 调试 2）。另外 assert 和 if 是分工的：assert 查“正常情况下必须成立”的内部条件，if 处理预期内会发生的业务分支，python -O 会跳过断言，所以它替代不了正式的输入校验。

三、实验仪器与编程环境

项 目	内 容
计算机	x64 架构，Windows 10 (10.0.19045)
Python	3.14.6，解释器路径由脚本用 Path(__file__) 自行定位，不写死
编程工具	VS Code / PowerShell 命令行，一条命令跑通全流程
第三方依赖	无。仅用标准库 re、unicodedata、csv、collections、pathlib；也未安装 jieba——指导书 3.6 节的 tokenize 本就是正则实现，零依赖方案换台电脑照样能复现
结果导出	DOCX 由脚本直接写 OOXML，PDF 由 reportlab 直出，全程不依赖 Word COM；正文宋体、标题黑体、代码新宋体都是系统自带

说明：本次实验全部代码不写死任何他人电脑的绝对路径，一律以 Path(__file__).resolve().parent 为基准定位 data/ 与 stopwords.txt。

四、实验内容

4.1 数据文件与个人任务

实验资源包目录为 实验一/ ├── expl_starter.py ├── stopwords.txt └── data/。A—D 为必做，E—J 为选做，选做不替代必做。本人完成了全部 A—D 必做组，并额外完成 E—J 全部 6 个选做组。

组	性质	文件名	主题	个人任务
A	必做	input_A_utf8.txt	健康数值与单位	保留血压、剂量单位、否定/约束词和段落
B	必做	input_B_utf8sig.txt	健康教育与隐私字段	掩码手机号和邮箱，同时保留血糖数值与单位
C	必做	input_C_gb18030.txt	康复训练与全角字符	全角数字/标点规范化，保留日期、时间与检查术语
D	必做	input_D_utf16.txt	检查记录与否定表达	停用词处理后仍保留 无/未/不伴 等否定表达

4.2 处理流程

原始字节 → ① 多编码探测（含 BOM 检测） → ② NFKC 规范化 + 空白折叠 → ③ 隐私掩码（含命中计数） → ④ 规则切词（多字优先） → ⑤ 停用词过滤（否定保护） → ⑥ 词频统计 → ⑦ 断言核验 → ⑧ 结果写入独立目录 outputs/

- 原始文件不被修改，结果全写入 outputs/；每组先跑断言再落盘，语义一旦丢失，程序在写盘之前就中止。

4.3 必做组 A—D 结果摘要

组	识别编码	尝试次数	原始→规范化→掩码 →token 字符数	手机/ 邮箱	停用词删除	专项断言
A	utf-8	1	95 → 94 → 94 → 57	0 / 0	与/患者×2	数值/单位/否定词/ 段落均保留
B	utf-8-sig	1	112 → 111 → 108 → 60	1 / 1	与/患者	手机与邮箱各命中 1 处，血糖保留
C	gb18030	3	96 → 95 → 95 → 65	0 / 0	与/患者	全角转半角，日期 时间与术语保留
D	utf-16	4	84 → 83 → 83 → 58	0 / 0	与/建议/患者×2/进行	过滤后 无/未/不伴 仍保留

4.4 【选做】选做组 E—J 结果与必做组的差异

选做组文件：optional_E_utf8.txt、optional_F_gb18030.txt、optional_G_utf8sig.txt、optional_H_utf16.txt、optional_I_utf8.txt、optional_J_gb18030.txt。六组我都做了。

组	主题	识别编码	尝试 次数	原始→规范化→掩码→token	规范化变化	否定 词留 存	段 落
E	公共卫生宣教与段落	utf-8	1	88 → 86 → 86 → 65	全角 4→0	—	是
F	心电检查说明	gb18030	3	85 → 84 → 84 → 58	全角 19→0	未	否
G	睡眠健康建议	utf-8-sig	1	91 → 90 → 90 → 52	全角 5→0	—	否
H	视力健康教育	utf-16	4	78 → 77 → 77 → 59	全角 4→0	未	否
I	用药安全科普	utf-8	1	82 → 81 → 81 → 61	全角 4→0	不得	否
J	老年健康管理	gb18030	3	81 → 80 → 80 → 63	全角 3→0	未	否

选做组多了两类必做组没有的考法：

- E 组考段落结构。两个连续空格被折叠成一个（原文“预防呼吸道传染病 应注意通风”），但两个语义段落之间的空行必须留着，所以 E 组字符数是 -2，其余 9 组都是 -1。
- G 组考“该改哪些、不该改哪些”：只统一正文大小写，标题组别【任务数据G】和编号 SIM-G 不动；实现上按行号切分，只对第 3 行往后小写化。断言查 caffeine 已出现、CAFFEINE 不再出现。另一条查 sim-g 没出现，看着矛盾其实不矛盾——SIM-G 只在第 2 行那行受保护的编号里，正文本来就没有它。J 组另外要求四阶段统计与编码错误记录落盘，见 outputs/阶段统计与编码诊断.csv。

4.5 【现场】新句任务卡：先预测、再运行、再用断言解释

按上机步骤 6，现场按“班级一名单序号”领一张新句任务卡，先写出预期处理结果，再运行程序，并解释一条与任务卡要求对应的断言。两个班各 30 张、共 60 张，我逐张核验；断言的期望值一律由该卡自己的输入句推导，不写死常量——手机号期望命中数取自输入句里 11 位数字串的个数，全角数字期望数取自输入句实际含有的全角数字个数，换一张卡不用改代码，也不会出现“断言写死了自己读出来的内容”这种自欺。60 张共 286 条断言，全部通过（A 组 16 张、B 组 16 张、C 组 14 张、D 组

14 张)。下面以我抽到的 1班-07 (C 组) 为例走完三步:

- 任务卡输入句: 康复训练: 上肢抬举 2 7 次。2 0 2 6 — 1 1 — 2 7 1 6 : 3 0 完成检查。
- ① 运行前写下的预期: 句中 14 个全角数字 (2 7、2 0 2 6、1 1、2 7、1 6、3 0) 全部转成半角; 全角冒号「:」与连字符「—」一并转半角; 「康复训练」「完成检查」等汉字一个都不能少; NFKC 是 1:1 映射, 所以字符总数不该变 (实测前后都是 34 个字符)。
- ② 程序的规范化输出: 康复训练:上肢抬举27次。2026-11-27 16:30完成检查。
- ③ 与任务卡要求对应的断言: 「C 专项: 输入含 14 个全角数字, 规范化后 0 个」。这条断言查的是规范化之后还能不能再找到一个全角数字——任务卡要求“规范化全角字符”, 它正好是这条要求的机械化写法。配套另两条查半角数字是否都已出现、汉字是否被误删, 三条全 PASS, 说明预期与实际一致。

预测与实际不一致之处: 批核 D 类卡片时曾整片报红“否定词全部丢失”。原因不是停用词表, 而是我把「未见」当成了整词, 切词后 token 里就没有「未」, 而指导书第 6 条断言恰恰要求 token 里含「未」。按指导书口径拆成 未 + 见 之后, D 类 14 张全部通过 (口径理由见 2.4)。

五、代码 (附注解) 与结果分析

5.1 关键代码片段

代码 1 多编码探测与 BOM 处理 (指导书 3.2 节, 并按实验步骤 2 补充 BOM 检查)

```
def read_text_checked(path):
    failed = []
    for encoding in CANDIDATE_ENCODINGS:
        try:
            text = Path(path).read_text(encoding=encoding)
        except UnicodeError as error:
            failed.append(("encoding": encoding, "error": type(error).__name__))
            continue
        # utf-8 能解码但残留 BOM —— 说明文件其实是 UTF-8-SIG, 改用 utf-8-sig 重读
        if text.startswith("\uffff"):
            try:
                text = Path(path).read_text(encoding="utf-8-sig")
            except UnicodeError:
                pass
            return text, "utf-8-sig", failed, "utf-8 可解码但首字符含 BOM, 已改用 utf-8-sig 重读"
        return text, encoding, failed, "无 BOM"
    raise UnicodeError(f"无法识别文件编码: {path} (已尝试 {CANDIDATE_ENCODINGS})")
```

代码 2 隐私字段掩码 (指导书 3.4 节)

```
PHONE_RE = re.compile(r"(?!\\d)(1[3-9]\\d{4})(?!\\d)")
EMAIL_RE = re.compile(r"([A-Za-z0-9._%+-]{2})[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+")
def privacy_mask(text):
    """
    掩码手机号与邮箱, 并返回各自的替换次数 (用于断言核验)。
    手机号 13912345671 → 139****5671 (前后加 (?!\\d)/(?!\\d), 避免从长数字中截取片段)
    邮箱 studentB@example.com → st***@example.com (保留前 2 个字符, 便于人工核对归属)
    说明: 本实验掩码只作用于合成教学数据, 不是经过临床验证的完整脱敏系统。
    """
    text, mobile_n = PHONE_RE.subn(r"1****2", text)
    text, email_n = EMAIL_RE.subn(r"1***2", text)
    return text, {"mobile": mobile_n, "email": email_n}
```

代码 3 断言记录与 D 组专项断言

```
# 每条断言记录为字典, 显式携带 group / kind / idx,
```

```
# 汇总时直接取字段，不再从标签字符串里反解组号（早期版本正是栽在这里）。
def check(group, kind, idx, text, condition, message):
    ok = bool(condition)
    label = f"{kind}[{group}{idx}] {text}"
    ASSERT_LOG.append({
        "group": group, "kind": kind, "idx": idx,
        "label": label, "ok": ok, "message": message,
    })
    assert ok, f"{label} 失败: {message}"
# ----- D 组专项：停用词过滤后否定表达仍保留 -----
need = ("无", "未", "不伴")
missing = [x for x in need if x not in res["tokens"]]
check(g, "专项", 1, "过滤停用词后 否定词 无/未/不伴 仍保留",
    not missing,
    f"D 组否定表达丢失: {missing}")
```

5.2 运行结果

python expl_starter.py — 断言汇总（31 条全部通过）

【断言汇总】

断言总数 31 条，通过 31 条，失败 0 条

含专项断言的数据组: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']

A-D 每组专项断言条数: A=1, B=2, C=1, D=1

结果已写入: D:\深技大课程学习\AI time\自然语言处理 实验一\实验一\outputs

实验一主流程完成: 10 组数据, 31 条断言全部通过。

图 1 断言汇总：10 组数据共 31 条断言，全部通过

5.3 结果分析

- (1) 编码识别。
- D 组 UTF-16 要试满 4 次 (utf-8、utf-8-sig、gb18030 全抛 UnicodeDecodeError)；C、F、J 组 GB18030 要试 3 次；A、E、I 组第一个候选就中。失败记录看着没用，其实正好说明“为什么最后选中的那个编码是对的”。
 - utf-16 几乎不抛异常，它把任意偶数长度字节强行两两配对。E、I 两个 UTF-8 文件被 utf-16 读也会“成功”，结果是脢 誢 跽 藥 铍 这样的乱码。所以我多打了一个汉字占比当可读性诊断（正常文本 46% - 78%，乱码远低于此），另外用“解码后不含 U+FFFD”兜一道硬断言。
- (2) 规范化前后对比（实测）。

组	规范化前	规范化后	关键变化
C	康复训练：上肢抬举 3 0 次。2 0 2 6 — 1 0 — 1 5 0 9：3 0 完成动态心电图检查	康复训练:上肢抬举30次。 2026-10-15 09:30完成动态心电图检查	全角数字、连字符、冒号、逗号全部转半角
A	患者无头痛，不伴胸痛。家庭血压 128/78 mmHg。	患者无头痛,不伴胸痛。家庭血压 128/78 mmHg。	全角逗号转半角；数值与单位原样保留

组	规范化前	规范化后	关键变化
E	预防呼吸道传染病 应注意通风、手卫生。	预防呼吸道传染病 应注意通风、手卫生。	两个连续空格折叠为一个，段落空行保留

这里有个坑：字符数变化不能用来判断规范化有没有生效。NFKC 的全角转半角是 1:1 映射，字符总数不动——A 组和 C 组分别把 5 个、22 个全角字符全转成半角之后，字符数照样只比原文少 1，那 1 个来自首尾 strip()。只盯字符数会得出“规范化没起作用”的相反结论，得逐个比对码位才行。

(3) 掩码与停用词。

B 组实测：手机号 13912345671 → 139****5671（命中 1 处，长度不变），邮箱 studentB@example.com → st***@example.com（命中 1 处）。整条文本 111 → 108，净减的 3 个字全来自邮箱变短，血糖 6.2 mmol/L 完好。脱敏不能顺手把业务数据一起删掉，这是 B 组专项断言要守的。

D 组切词后删掉的停用词是 与/建议/患者×2/进行，「无、未、不伴」全部存活。「未见异常放电」被切成 未+见 两个 token，否定意思由「未」扛着，没丢；反过来要是把「未见」当整词，token 里就没有「未」，指导书第 6 条断言会先失败。

(4) 词频。

A 组 Top-10 是 数×2、A×2、为×2、痛×2，其余都是 1 次；C 组是 训×3、练×3、C×2、康×2、复×2、成×2、不×2、30×2。每篇文本才 3-5 行、78-112 字符，单字切完词表很稀疏，高频的基本都是重复的字。规则切词在这种小样本上够用，但它切出来的不是词，「训练」「康复」都成了单字。

5.4 调试记录：报错现象—原因定位—修改—复测

实验过程中共记录 4 个真实错误，均可用 debug_record.py 复现。

编号	报错现象	原因定位	修改	复测
调试 1	断言 AssertionError: B 组首行应以【任务数据B 开头，实际为 '\uffeff【任务数据B: 健康教育'	B 组是 UTF-8-SIG。带 BOM 的 UTF-8 字节序列仍是合法 UTF-8，所以 utf-8 解码不报错，但首字符残留 U+FEFF。属“成功解出脏数据”	解码成功后检查首字符是否为 BOM，命中则改用 utf-8-sig 重读	通过。识别编码由 utf-8 改判为 utf-8-sig，首行干净
调试 2	断言 AssertionError: D 组否定表达丢失: ['无', '未', '不伴']	用一份随手扩大的停用词表过滤，其中含无、未、不伴三个否定词，“未见异常放电”被清洗成“见异常放电”，语义直接反转	load_stopwords 加载后减去否定保护词 PROTECT_WORDS	通过。实际只删除与/患者/建议/进行，否定词全部存活
调试 3	直接照抄指导书 3.5.1 节第 3 条样例断言，AssertionError: A 组语义信息丢失	样例断言写的是“5 mg/片”（带尾随空格），但原文“5 mg/片”后紧跟全角逗号 U+FF0C，根本不存在空格。指导书排版留白被 PDF 抽文本保留了下来	断言只写确定存在的子串“5 mg/片”，空格不入断言	通过。这说明断言的期望值必须来自实测数据，不能来自文档排版
调试 4	31 条数据断言全部 PASS，程序却崩在 AssertionError: A 组缺少专门断言	汇总处用 l.split(" ")[1].split(" ")[0] 从标签文本反解组号，但通用标签是“通用[A]”、专项标签却是“专项[1]”，格式不一致导致匹配不到。真正的数据处理全部正确，崩溃发生在自检层	check() 改为显式接收 group/kind/idx 并存入字典，汇总直接取字段	通过。A 组专项断言条数统计为 1，不再误报

六、实验总结与感悟（AI辅助说明）

6.1 总结与感悟

做完印象最深的一点，是这条链路上每一步都可能悄悄改掉语义，而程序不会吭声。

- 没报错不等于做对了。utf-16 能把任意偶数长度字节解成“看起来挺正常”的乱码，utf-8 能读开带 BOM 的文件却留下 U+FEFF，两个都不抛异常。是靠 BOM 检测、替换字符检测、汉字占比这些主动检查才拦住的。
- 洗得越干净，风险越大。为了省事直接用“只保留汉字”一步到位，代码是短了，可 128/78 mmHg 和 5 mg/片 全没了。医疗文本里删掉一个数值和写错一个数值，后果是同一量级的。否定词同理，把 未/无/不伴 塞进停用词表，等于把句子里的“是”删掉。
- 断言是把“我以为对”变成“程序验过对”的东西。31 条全过之前，结果都只是看着合理。这次 4 个坑里有 3 个出在我自己的代码上——自检逻辑写错、期望值写死、切词口径想当然，只有 1 个来自指导书样例。

6.2 AI 辅助说明

指导书要求写明工具/模型、使用目的、关键提问概述、采用内容及人工核验；模板另要求体现使用环节、具体交互、验证与核对、收获与反思。两处合并为七项：

项 目	说 明
工具与模型	MiniMax Code（桌面版），模型 MiniMax-M3.1-Flash-Preview，使用日期 2026-10-09。实验与报告都在本人电脑上完成。
使用目的	查资料、搭初稿，以及编码探测与排版这类要反复试的活。它不替我运行程序、不看我的 data，报告里的数字仍以本机输出为准。
使用环节	读指导书与模板；写 expl_starter.py 流水线；定位并修复运行报错；跑通 60 张现场任务卡共 286 条断言；按模板写报告并排版为 DOCX 与 PDF。
具体交互与关键提问概述	① “依据参考资料完成实验并制作报告”； ② “A 组断言报语义信息丢失”——打印码位后确认「5 mg/片」后是全角逗号 U+FF0C 而非空格，AI 沿用的样例多带了尾随空格； ③ “31 条断言全 PASS 为何反报缺专项断言”“D 类任务卡为何全报否定词丢失”，定位到 split("[")[1] 反解组号、以及「未见」须拆成「未」+「见」。
采用内容	多编码探测与 BOM 检查的思路、NFKC 流程、正则掩码并用 subn 统计替换次数、否定保护词、断言分层组织、报告章节骨架与排版参数。
验证与核对	AI 产出一律当假设，没跑过的不写进报告：每个数字都由本机跑出来，可用 verify_report.py 与 outputs/ 产物比对，88 项检查全过；「5 mg/片」和把「未见」当整词这两处期望值都是核对后改的。
收获与反思	思考题里有两条断言是我第一版凭常识写的，实测后一条被推翻、一条引用的字段根本不存在——这类错误只有真跑才会发现。

七、思考题

思考题 1 为什么不能把所有数字、英文和标点全部删除？结合个人数据举例。

数字和英文是语义主体，删不得。把数字和字母全去掉，「家庭血压 128/78 mmHg」只剩「家庭血压 /」，「科普标签为5 mg/片」只剩「科普标签为 /片」，「空腹血糖为6.2 mmol/L」退成「空腹血糖为. /」——数值、剂量、单位全没了，剩下的已经不是病历。

标点同样不能删，它的问题只是全角半角混用，所以本实验做的是 NFKC 统一字形，不是删字符。删掉 D 组的逗号，「脑电图检查未见异常放电，患者无意识障碍，不伴明显头痛」粘成「脑电图检查未见异常放电患者无意识障碍不伴明显头痛」，否定管到哪一句就读不出来了；C 组日期原本是全角，归一后是 2026-10-15 09:30，删掉连字符和冒号只剩 20261015 0930。

思考题 2 errors="ignore" 可能造成什么问题？

危害是让解不开的内容凭空消失，而程序和断言都不吭声。C 组原始 182 字节，utf-8 硬解在第 0 字节就抛 UnicodeDecodeError；加上 errors="ignore" 之后解码“成功”，只剩 35 个字符，80.8% 被静默丢掉。

它麻烦的地方是能骗过检查。指导书第 2 条通用断言查的是“不含 U+FFFD”，而 ignore 是丢弃不是替换，结果里根本没有替换字符，断言照样通过；换成 errors="replace" 反而留下 118 个 U+FFFD，当场被拦。最后是 C 组“2026-10-15 09:30 必须保留”那条数据断言碰巧拦住的，可它只拦日期。脱敏和规范化可以有损，解码不行。

思考题 3 NFKC 规范化可能带来哪些收益和风险？

收益有三：全角半角统一，正则不用写两套；“2026”和“2026”算同一个字符串，检索可靠；不同来源系统带来的字符差异被抹平。

风险也有三。有损且不可逆；它连标点一起动，实测 C 组的「，」「：」全转成半角（C 组无分号，分号在 B 组），而中文习惯的“。”（U+3002）不动，标点风格就中西混了；兼容分解还会改写看着无关的字符，我实测 ① 会变成 1、(→) 会变成 (一)、m² 会变成 m2。所以规范化只在检索统计环节用，原始文本另存一份，不原地覆盖。

思考题 4 隐私正则可能产生哪些漏报或误报？

漏报方面，带连字符的 138-1234-5678、空格分隔的 139 1234 5671、反垃圾写法的 studentB[at]example[dot]com 都匹配不上；微信、身份证、银行卡这些字段本实验没覆盖。

误报更具体：一串 18 位数字 20261015000012345678 会被掩成 2026101****012345678，中间 4 位被当成手机号——正则只管形似，不管真假。反过来 C 组归一后的 2026-10-15 09:30、A 组的 5 mg/片、B 组的 6.2 mmol/L 都没被误伤，靠的是前后数字断言和掩码后长度不变这两条限制。所以正则脱敏只是第一道防线。

电子签名：

成绩评定：